Facultad de ingeniería

Manejo de archivos

Docente: ING. Rojas Vásquez Manuel Gustavo

# Manual técnico proyecto 2

Nombre: Davis José Alexis García López

Carne: 1554725

Quetzaltenango 9 de septiembre del 2026

# Instrucción Y Arquitectura Del Sistema

El proyecto **Agenda de Contactos** es una solución desarrollada en Python 3.13 enfocada en la gestión eficiente de almacenamiento físico persistente. Modela el comportamiento de bajo nivel de los sistemas de archivos mediante estructuras binarias de longitud fija e índices dinámicos en memoria secundaria.

La arquitectura del código se distribuye modularmente bajo el siguiente esquema de paquetes:

- `main.py`: Punto de entrada principal que arranca la aplicación e inicializa las estructuras físicas.
- `src/modelos/contacto.py`: Clase de dominio `Contacto` encargada del empaquetado binario y parseo de bytes UTF-8.
- `src/logica/lectura_escritura.py`: Operaciones I/O de bajo nivel con punteros `seek()` y `tell()`.
- `src/logica/indice.json`: Indice primario de acceso directo O(1).
- `src/logica/sha256.py` y `zip.py`: Módulos de auditoría de seguridad y respaldos comprimidos.
- `src/exportadores/`: Generación de reportes externos legibles en JSON y PDF.
- `src/interfaz/`: Capa de presentación construida sobre Tkinter/TTK.

## 1. Modelado y Estructura de Registros Binarios (100 Bytes)

Para lograr acceso aleatorio sin lectura secuencial, cada registro se empaqueta de forma estricta en **100 bytes UTF-8**. Si los datos contienen caracteres especiales (como tildes o `ñ`), el formateo calcula la longitud en *bytes reales* y no en caracteres simples.

| Campo | Tipo de Dato | Rango en Bytes | Longitud Fija |
|---|---|---|---|
| **ID Contacto** | Cadena (UTF-8) | Bytes 0 a 9 | 10 Bytes |
| **Nombre Completo** | Cadena (UTF-8) | Bytes 10 a 39 | 30 Bytes |
| **Teléfono** | Cadena (UTF-8) | Bytes 40 a 54 | 15 Bytes |
| **Email** | Cadena (UTF-8) | Bytes 55 a 99 | 45 Bytes |
| | | **Tamaño Total del Registro:** | **100 Bytes** |

## 2. Lógica I/O con Punteros en Disco ( `seek` / `tell` )

La ubicación binaria exacta de cualquier registro N dentro del archivo `contactos.dat` se calcula mediante la ecuación matemática de desplazamiento lineal:

> Fórmula de Posicionamiento Físico:
>
> **Posición en Bytes = Número de Registro × 100**

Para realizar actualizaciones *in situ* (sobrescribir un registro sin reescribir todo el archivo), se posiciona el puntero mediante `f.seek(posicion)` en modo de apertura binaria `r+b`:

```
def escribir_registro(contacto, num_registro=None):
    inicializar_archivo()
    indice = cargar_indice()
    id_str = contacto.id.strip()

    if id_str in indice:
        num_registro = indice[id_str]  # Posición existente
    elif num_registro is None:
        num_registro = len(indice)
        indice[id_str] = num_registro
        guardar_indice(indice)

    posicion = num_registro * 100
    registro_bytes = contacto.a_registro()

    with open('data/contactos.dat', 'r+b') as f:
        f.seek(posicion)         # Salto directo en disco O(1)
        f.write(registro_bytes)  # Sobrescritura exacta de 100 bytes

    return num_registro, True
```

## 3. Gestión del Índice Primario JSON

El mapa primario conservado en `data/indice.json` desacopla las búsquedas del barrido secuencial en disco, ofreciendo una complejidad temporal `O(1)`:

```
{
  "CT001":  0,
  "CT002":  1,
  "CT003":  2,
  "CT004":  3,
  "CT005":  4,
  "CT006":  5
}
```

## 4. Módulos de Seguridad y Respaldo

### 4.1 Verificación Hash SHA-256

El módulo `sha256.py` garantiza la inalterabilidad de la base de datos binaria procesando el flujo de datos en bloques de 8KB mediante el algoritmo criptográfico SHA-256:

```
def calcular_hash(archivo_ruta='data/contactos.dat'):
    sha256_hash = hashlib.sha256()
    with open(archivo_ruta, 'rb') as f:
        for bloque in iter(lambda: f.read(8192), b''):
            sha256_hash.update(bloque)
    return sha256_hash.hexdigest()
```

### 4.2 Compresión ZIP

El módulo `zip.py` utiliza la librería estándar `zipfile` para empaquetar de forma atómica los archivos de persistencia ( `contactos.dat` e `indice.json` ), retornando el porcentaje exacto de ahorro en disco.

## 5. Tabla de Complejidad Computacional (Big-O)

| Operación | Complejidad Temporal | Explicación Técnica |
|---|---|---|
| **Búsqueda por ID** | `O(1)` | Lectura directa en la tabla Hash/Índice JSON. |
| **Lectura de Registro** | `O(1)` | Posicionamiento mediante `seek(N * 100)` y lectura de 100 bytes. |
| **Actualización In Situ** | `O(1)` | Sobrescritura binaria directa sin desplazamiento de registros. |
| **Eliminación de Registro** | `O(N)` | Reconstrucción del buffer binario y reindexación de posiciones. |
| **Cálculo Hash SHA-256** | `O(N)` | Recorrido completo en bloques de 8KB sobre el tamaño total del archivo. |

## 6. Conclusiones del Desarrollo Técnico

1. El diseño de registros de longitud fija (100 bytes) combinado con la codificación de caracteres a nivel de *bytes UTF-8 reales* previene la desalineación o corrupción de punteros al manipular texto con tildes o caracteres especiales.
2. El patrón de indexación JSON desacopla la capa de presentación Tkinter de la IO de disco, permitiendo búsquedas inmediatas en `O(1)`.
3. Las firmas criptográficas SHA-256 proveen un mecanismo eficaz de auditoría que detecta cualquier inconsistencia o modificación no autorizada externa en el archivo `contactos.dat`.